

Quantitative Architecture & Model Evaluation

End-to-End Trading Pipeline, Quantitative Models & Parameter Sweeps

- **Repository:** `fts`
- **Target Asset Class:** High-Frequency / Crypto Time Series (e.g. 30-min BTC)
- **Core Stack:** Python, PyTorch, ONNX, SQLite, Optuna, Custom Event-Driven Backtester

1. System High-Level Workflows

The framework is partitioned into **3 decoupled quantitative execution loops**:

```
+-----+
| Workflow 1: Model Optimization & Selection (02_strategy_hparam_training) |
| MarketDataRepository -> DatasetBuilder -> Optuna Engine -> ModelRepository (ONNX) |
+-----+
|
| v
|
+-----+
| Workflow 2: Event-Driven Single Backtest (03_custom_backtest) |
| ReplayLoop -> TradingPipeline -> StrategyEngine -> RiskManager |
| -> ExecutionEngine (Delay/Slippage) -> BacktestVisualizer |
+-----+
|
| v
|
+-----+
| Workflow 3: Multi-Run Parameter Sweep (04_parameter_sweep_backtest) |
| SweepSpec Grid -> Parallel Backtest Runner -> Aggregate Heatmaps & Visualizers |
+-----+
```

2. Domain Architecture I-A: Data Management & Feature Pipeline

Historical Data Retrieval

- **MarketDataRepository**
 - Serves as the single source of truth for historical market data retrieval.
 - Abstracts database queries to provide clean, windowed OHLCV time-series data while guaranteeing point-in-time consistency.

Feature Transformation & Dataset Construction

- **DatasetBuilder & Feature Transformation Pipeline**
 - Converts raw historical market bars into stationary machine learning feature datasets (X) and forward return targets (y).
 - Handles rolling windowing, sequence creation, feature standardization, and non-overlapping train/validation splits without look-ahead leakage.

3. Domain Architecture I-B: Model Registry & Signal Classification

Model Asset Catalog & Specs

- **HParamStudySpec & ModelRepository**
 - Defines hyperparameter search spaces for Optuna optimization studies.
 - Serves as model asset registry storing compiled ONNX weights, feature specs, input schemas, and out-of-sample performance score logs.

Dynamic Signal Classification Rule Layer

- **DynamicThresholdClassifier**
 - Decision rule layer translating raw model predictions into discrete directional trade signals (Long / Short / Flat).
 - Uses adaptive probability thresholds (e.g., dynamic confidence quantiles) to filter out regime noise and execute high-conviction trades only.

4. Domain Architecture II-A: Signals & Strategy Engine

Real-Time Feature Orchestration

- **TradingPipeline**
 - Stateful real-time feature orchestrator maintaining rolling lookback buffers across continuous bar feeds.

Strategy Implementation

- **NetsStrategy**
 - Quantitative strategy class integrating live feature transforms, neural network inference, and dynamic signal classification rules.

Strategy Coordination

- **StrategyEngine**
 - Master coordinator transforming directional alpha signals into target portfolio position allocations across multiple strategies.

5. Domain Architecture II-B: Portfolio & Risk Management

Account Accounting

- **Portfolio**
 - Real-time ledger tracking available cash balance, gross/net position exposure, unrealized/realized PnL, and contract holdings.

Position Allocation

- **FixedPercentageSizer**
 - Position sizing algorithm determining order contract counts based on a fixed percentage of current portfolio equity.

Risk Control Gateway

- **RiskManager**
 - Evaluates proposed strategy trades against account constraints (max position size, drawdown limits, leverage limits) to enforce risk guidelines.

6. Domain Architecture III-A: Execution Simulation & Latency

Latency Modeling

- **KBarExecuteDelay**
 - Simulates execution latency by delaying order fill matching by K bars (signal at Bar N close is filled at Bar $N + 1$ open), preventing look-ahead bias.

Market Impact & Slippage

- **FlatPriceSlip**
 - Slippage model simulating market impact and bid-ask spread costs by applying fixed or ratio-based price adjustments to fill prices.

Virtual Exchange Engine

- **SimulatedExecutionHandler & ExecutionEngine**
 - Virtual exchange matching engine applying delay/slippage rules, tracking commissions, and issuing execution fill notifications.

7. Domain Architecture III-B: Replay & Parameter Sweeps

Event Loop Dispatcher

- **HistoricalReplayLoop**
 - Drives time forward tick-by-tick or bar-by-bar during backtests in strict chronological order.

Parameter Sweeps & Analytics

- **SweepVisualizer** & Exporters
 - Visualizes cross-run parameter sensitivity heatmaps and metric tradeoffs across multi-run backtest grids.
 - Exports self-contained interactive dashboards for multi-parameter robustness evaluation.

8. Quantitative Model Comparison Matrix

Characteristic	Linear Regression	XGBoost	LSTM (Deep Learning)
Model Type	Linear Parametric	Gradient Boosted Trees	Recurrent Neural Network
Input Requirements	Requires strict stationarity	Handles raw tabular features	Requires normalized sequences
Non-Linear Dynamics	✗ None (linear only)	■ High (step function split)	■ High (smooth non-linear)
Temporal Memory	✗ None (lag columns required)	✗ None (lag columns required)	■ Native cell state buffer
Overfitting Risk	■ Low (high bias)	⚠ High (fits noise tails)	⚠ High (high variance)
Inference Latency	⚡ Sub-millisecond ($< 1\mu s$)	⚡ Fast ($< 1ms$)	🕒 Moderate ($1--10ms$)
Outlier Resilience	⚠ Moderate (OLS squared penalty)	■ Robust (tree quantiles)	✗ Poor (exploding gradients)

9. Quant Deep Dive: Linear Regression vs XGBoost

Linear Regression (Baseline Benchmark)

- **Strengths:** Closed-form solution ($O(N)$), transparent feature weights, minimal risk of overfitting noisy financial data, sub-microsecond inference speed.
- **Weaknesses:** Cannot model regime shifts, volatility clustering, or non-linear tail interactions. Fails when y is governed by threshold logic.

XGBoost (Gradient Boosted Decision Trees)

- **Strengths:** Superior on tabular feature sets (e.g. order book imbalances). Invariant to monotonic scaling (no `StandardScaler` needed). Captures non-linear feature interactions naturally.
- **Weaknesses:** Completely blind to sequential order unless explicit lag columns ($t - 1, t - 2$) are engineered. Poor at extrapolating beyond historical min/max feature bounds.

10. Quant Deep Dive: LSTM Architectural Logic & Dual Recurrence

Dual Recurrence Mechanism

- **Internal Cell State (C_t)**: Acts as a **Constant Error Carousel (CEC)** that propagates long-term memory across sequence steps without vanishing gradients.
- **Outer Hidden State (h_t)**: Filters and exposes short-term memory to output predictions and upper network layers.

Gate Functions & Memory Operations

- **Forget Gate**: Evaluates previous hidden state and new inputs to decide how much past cell memory to keep or discard.
- **Input Gate & Candidate State**: Determines which new information from recent market bars should be written into the cell state.
- **Output Gate**: Controls how much of the updated internal cell state is exposed to form the outer hidden state.

11. Quant Deep Dive: LSTM Strengths in Quant Finance

End-to-End Sequence Memory

- Retains historical lookback cell state natively across variable horizons without requiring manual lag feature expansion.

Regime & Pattern Recognition

- Captures complex multi-step market momentum, volatility regime transitions, and non-linear mean reversion patterns.

Smooth Non-Linear Interactions

- Fits continuous non-linear decision surfaces across price return inputs, outperforming rigid linear approximations.

12. Quant Deep Dive: LSTM Vulnerabilities in Quant Finance

Signal-to-Noise Ratio (SNR) Sensitivity

- High network capacity leads the model to memorize noise patterns and spurious correlations in financial return series.

Gradient Instability from Return Spikes

- Extreme volume or price return outliers generate massive gradient updates that disrupt learned internal cell weights.

Overfitting & Optimization Complexity

- Highly sensitive to hyperparameter tuning; requires strict regularization (Dropout, Weight Decay) and non-overlapping embargo splits.

13. Empirical Case Study: Experimental Setup & Splits

Dataset Timeline & Rationale

- **Train / Validation Split:** 2025-11-04 19:30:00 UTC to 2025-05-30 19:30:00 UTC
 - Used for hyperparameter optimization (Optuna) and model training.
- **Embargo Split:** 2025-05-30 19:30:00 UTC to 2026-06-01 03:30:00 UTC
 - **Purposely Purged Window:** Prevents serial correlation overlap and look-ahead leakage between training and testing data.
- **Test Split:** 2026-06-01 03:30:00 UTC to 2026-06-21 23:00:00 UTC
 - Clean out-of-sample evaluation window on 30-minute BTC data.

14. Empirical Case Study: 1D vs 5D Feature Results

Empirical Findings (LSTM Models on 30m BTC)

- **Model 29 (1D Feature Input: Close Log Returns only)**
 - Validation Information Coefficient (IC): **0.137** ●
- **Model 48 (5D Feature Input: Close Return + Volume + High/Close + Low/Close)**
 - Validation Information Coefficient (IC): **0.107** ●

Key Finding: Moving from a simple 1D return series to a 5D multi-feature pipeline resulted in a **22% drop in validation Information Coefficient (IC)**.

15. Theoretical Diagnosis: Why More Features Hurt

```
+-----+
| 1. Signal-to-Noise Ratio (SNR) Dilution |
| Close return holds strong autocorrelation. Volume & ratio tails inject |
| high-frequency noise that dilutes the primary predictive signal. |
+-----+
| 2. Wildly Mismatched Feature Distributions |
| Close returns are symmetric near 0. High/Close ( $\geq 0$ ) and Low/Close ( $\leq 0$ ) |
| are asymmetric. Volume change has extreme fat-tailed spike outliers. |
+-----+
| 3. Curse of Dimensionality & Overfitting |
| Input dimension 1 -> 5 increases first-layer weights. On ~6,000 samples, |
| the network overfits noisy volume patterns, degrading validation IC. |
+-----+
| 4. Hyperparameter Suboptimality |
| Search space tuned for 1D sequences lacked sufficient dropout & weight |
| decay needed to regularize higher-dimensional input spaces. |
+-----+
```

16. Backtesting Realism & Parameter Sweeps

Execution Realism

- **Latency Simulation:** `KBarExecuteDelay(delay=1)` delays order fills by 1 bar (Signal at Bar N close $\rightarrow$ Execution at Bar $N + 1$ open).
- **Slippage & Costs:** `FlatPriceSlip` incorporates bid-ask spread and market impact penalties.

Interactive Parameter Sweep Reports

- Multi-run parameter grid sensitivity dashboards:
 -  XGBoost Max Depth Sweep
 -  Hidden Dimension Sweep
 -  LSTM Number of Layers Sweep

17. Quantitative Research Roadmap & Next Steps

1. Target Realism

Align training target y_t to **2-bar forward returns** (r_{t+2}) when running backtests with a 1-bar execution delay.

2. Latency-Aware Loss

Incorporate non-fill probabilities, execution latency distributions, and slippage penalties directly into PyTorch loss functions.

3. Volatility Forecasting

Deploy fast volatility sub-models to dynamically scale position sizing and confidence threshold rules across market volatility regimes.

4. Model Ensembles

Implement `PortfolioStrategy` to combine decoupled single-feature (1D) and multi-feature (5D) models into an ensemble portfolio.